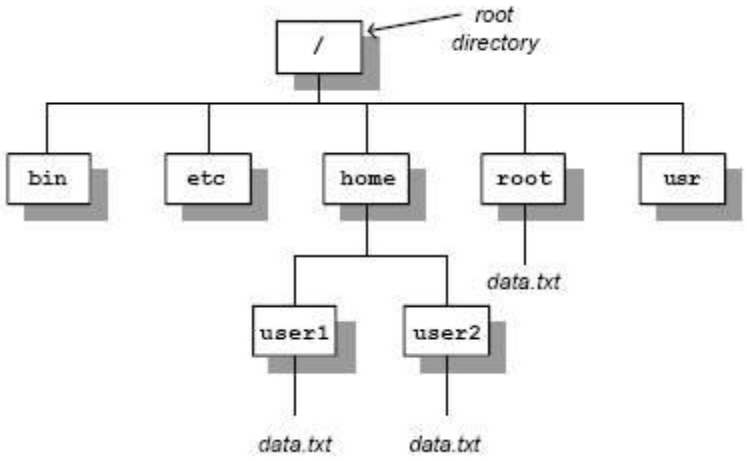
Linux 명령어, vi 편집기, Makefile

유닉스 프로그래밍

기본 개념

- **쉘**
  - 사용자를 위한 인터페이스를 제공해주는 소프트웨어
  - 그래픽 쉘과 텍스트 쉘로 나뉨

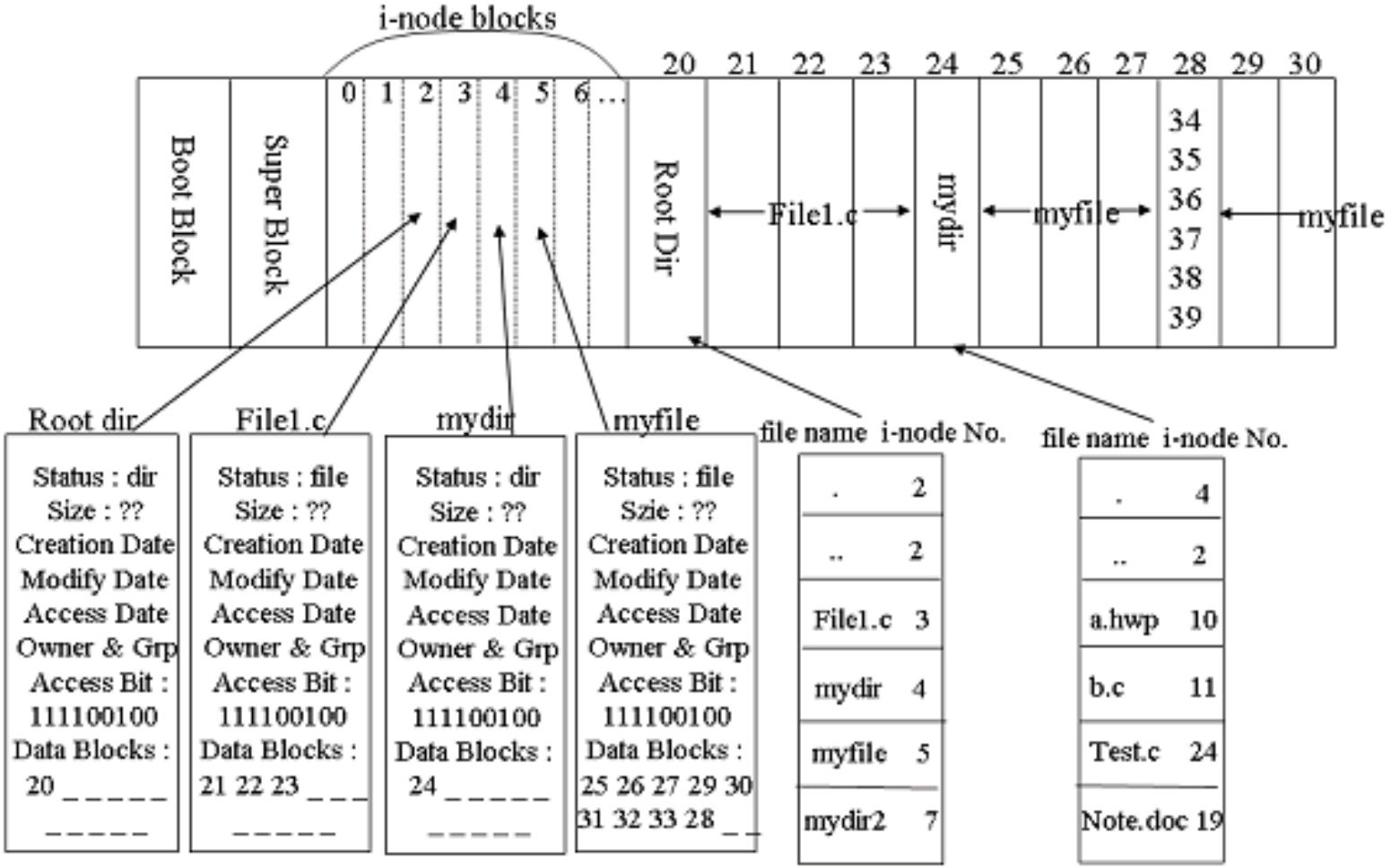
- **리눅스의 디렉토리(폴더) 계층 구조**



파일시스템

- **파일시스템**
  - 저장장치에 파일 형태로 데이터를 저장하는 운영체제 내부의 코드
  - 디스크를 파티션하여 파티션마다 파일시스템 포맷
  - Linux 파일시스템: ext4 파일 시스템
- **Linux 파일시스템 주요 구조**
  - Super-block
    - 디스크 물리적 정보, 파일시스템 내 i-node 개수, 총 데이터 블록 개수 등 전체 파일시스템과 관련된 정보를 유지
  - Inode
    - 파일(또는 디렉토리)마다 대응되는 i-node가 존재
    - 파일(디렉토리)에 대한 (이름을 제외한) 모든 정보를 저장
      - 생성,변경,접근 날짜, 퍼미션, 크기, 종류, 데이타블록번호

Design of UNIX File System



파일시스템

- **일반파일(regular file)**
  - 데이터를 담고 있는 파일
    - 예: test.c myhomework.hwp
- **디렉토리 파일**
  - 디렉토리 내에 존재하는 파일이나 서브디렉토리 정보를 테이블 형태로 유지
  - Linux 파일시스템의 경우 (inode, 이름)으로 구성된 테이블 형태
  - “.”과 “..”은 자신과 부모 디렉토리를 가짐
  - 사용자가 로그인하면 홈디렉토리부터 시작

파일시스템

- **특수파일**
  - /dev 에 존재, 디바이스를 대표하는 파일
  - 문자특수 파일
    - 문자 디바이스를 대표하는 파일
  - 블록 특수 파일
    - 블록 디바이스를 대표하는 파일
- **소켓**
  - 컴퓨터 사이에 데이터 통신을 위한 인터페이스
  - 소켓을 만들고 로컬과 원격의 주소를 설정
- **네임드파이프**
  - 파일시스템에 이름을 부여하고 생성한 파이프 파일
  - 두 프로세스간 데이터 전달을 중개

Linux의 디렉토리 구조

디랙토리	내 용
/	가장 위쪽의 디렉터리는 루트 디렉터리(root directory)라고 하고 /(슬래쉬)로 표현한다.
/bin	binaries의 약자이며, 실행 파일들이 모여 있다. 이 디렉터리에는 필수적인 많은 프로그램들이 포함되어 있다.
/dev	/dev 안의 파일들은 디바이스 드라이버들이다. 이것들은 디스크 드라이버, 모뎀, 메모리 등과 같은 시스템 디바이스나 자원들을 액세스하는 데 사용된다.
/etc	시스템 설정 파일, 프로그램, 유틸리티 등 다양한 프로그램들이 있으며 대부분의 파일들은 관리자에 의해 사용되는 것이다. 시스템의 전반에 걸친 설정 파일들 및 초기 스크립트 파일들이 있다.
/home	/home은 일반 사용자들의 홈 디렉터리로 할당하여 사용하는 공간이다.
/lib	공유 라이브러리 이미지를 포함하고 있다. 이 파일들은 일반적으로 사용되고 많은 프로그램에서 호출되는 코드들을 포함하고 있다. 공유 될 수 있는 루틴을 단독으로 포함하지 않고, 그 루틴을 일반 장소인 /lib 안에 저장함으로서 많은 사용자들이 공유할 수 있도록 되어 있다.
/proc	가상 파일 시스템이다. 이 디렉터리의 내용들은 시스템에서 운영되고 있는 다양한 프로세서들에 관한 내용과 프로그램에 대한 정보를 포함하고 있다.
/tmp	어떤 프로그램들은 특정 데이터가 임시 파일 안에 저장되기를 원한다. 이런 데이터들이 위치하는 장소이다.

Linux의 디렉토리 구조

디랙토리	내 용
/usr	응용 패키지들이 설치되어 있는 디렉터리이다. 마이크로소프트 Windows의 Program Files 디렉터리와 같은 역할을 한다고 생각하면 이해하기 쉬울 것이다.
/opt	Add-On 소프트웨어 패키지가 설치된다.
/mnt	마운트 디렉터리 라고 하며, 각종 입출력장치와 연결할 때 사용한다. 마운트하지 않은 상태에서는 빈 디렉터리로 존재하지만 입출력장치를 마운트하게 되면 해당 파일 시스템의 내용이 그대로 이곳에 놓이게 된다.
/var	이 디렉터리 아래에는 시스템 작동 중 변경되는 파일들이 담겨 있다. 로그 파일이나 스풀 파일들이 그것들인데, 즉 다른 시스템과 공유가 되지 않음을 의미한다.
/lost+found	파일 시스템의 이상 유무를 진단하고 복구하는 프로그램인 fsck(FileSystem Check)에 의해서 사용되는 디렉터리이다. 손상된 파일이나 디렉터리를 /lost+found 디렉터리로 연결한 뒤에 오류를 수정하게 된다.
/media	시디롬이나 플로피 디스크 혹은 USB 메모리 등이 임시로 마운트 되는 디렉터리이다.
/boot	리눅스 시스템 부팅에 관련한 모든 파일이 포함되어 있다.
/sbin	Standalone bimary 디렉터리이며 시스템 운영에 필요한 명령어들을 포함하고 있다. 슈퍼유저만이 사용할 수 있는 halt, shutdown 등의 명령어들로 구성되어 있다.

경로 기본 개념

- **절대 경로**
  - 파일의 절대 적인 위치
- **상대 경로**
  - 현재 속해있는 위치로부터 파일의 상대적인 위치
- **/home/minsu/game/diablo3/play 파일에 접근할 수 있는 방법**
  - 현재 /home/minsu이라는 디렉토리에 속해있다 가정
  - 1. 상대 경로를 통해 접근
    - game/diablo3/play
  - 2. 절대 경로를 통해 접근
    - /home/minsu/game/diablo3/play
- **특별한 디렉토리 이름**
  - . ← 현재 디렉토리를 의미
  - .. ← 상위 디렉토리를 의미
  - ~ ← 사용자의 홈 디렉토리를 의미

Linux 명령어

리눅스 명령어

- **Command line**

```
$ ls
a.txt          myfile.c          embedded_system
```

- **옵션: '-' 로 시작**

```
$ ls  -l
total 24
drwxr-xr-x 2 dlee dlee 4096 7월 5 13:34 Desktop
drwxr-xr-x 2 dlee dlee 4096 7월 15 12:45 Papers
- rw-r--r-- 1 dlee dlee 1024 7월 15 10:27 notepad
-rwxrwxr-- 1 dlee dlee  1024 7월 15 10:26 homework.hwp
```

- **두 개 이상 명령 입력시 ';'으로 분리**

```
$ ls;  date
a.txt          myfile.c          embedded_system
Fri Sep  6 13:58:00 EST 2013
```

리눅스 명령어

- **포어그라운드(Foreground) 명령어**
  - 쉘이 명령을 실행시키고, 쉘 자신은 명령 실행이 종료될 때가지 대기
  - 새로 실행되는 명령이 터미널 입출력 권한을 가짐
- **백그라운드(Backgound) 명령어**
  - 명령 제일 뒤에 '&'를 붙임
  - 쉘은 백그라운드로 명령을 실행시키과 동시에 쉘 자신도 실행하여 다음 명령 입력을 대기
  - 쉘이 터미널 입출력 권한을 가짐
  - 백그라운드로 실행되는 명령은 출력은 가능하지만, 터미널 입력을 요청하면 중단되며, 포어드라운드가 될 때 깨어남

```
$ find  /  -name  *.doc  -print  &          ⇦ find 명령을 백그라운드로 실행
16384                                       ⇦ 프로세스 ID
$ date                                      ⇦ date 명령의 실행
Fri Sep  6 13:58:00 EST 2013
```

파일 관련 주요 Linux 명령어

- **ls (list)**
  - 디렉토리의 내용을 출력
- **cd (change directory)**
  - 현재 디렉토리를 바꿈
- **pwd (print working directory)**
  - 현재 작업 디렉토리 출력
- **cat**
  - 파일 생성 혹은 파일의 내용을 출력
- **mkdir (make directory)**
  - 디렉토리 생성
- **rmdir (remove directory)**
  - 디렉토리 삭제
- **rm (remove)**
  - 파일 삭제
- **cp (copy)**
  - 파일 복사
- **mv (move)**
  - 파일 이동, 파일 이름 변경

Linux 명령어: ls

- ls 명령 옵션

옵 션	내 용
-1	터미널에 파일이나 디렉터리를 한 줄씩 출력하게 한다.
-a	숨겨진 파일과 디렉터리까지 모두 출력한다.
-b	파일명이 비텍스트 문자인 것을 8진수로 출력한다(비텍스트라는 것은 한글도 속한다).
-c	-t 나 -l 형식으로 출력할 때, 파일 변경 시간이 아니라 i-node 변경시간을 사용하여 출력한다. i-node가 변경되는 시간은 파일의 모드를 변경할 때이다.
-C	복수 개의 칼럼을 이용하여 출력한다. 이것은 파일을 출력할 때의 기본적인 방법이다.
-d	지정한 디렉터리에 대한 정보만을 출력한다.
-f	출력할 목록에 대해 소트(Sort)를 하지 않고 출력한다. 이 옵션을 쓰면 -l, -t, -s, -r 옵션은 무시된다. 또한 -a 옵션이 자동으로 사용된다.
-F	파일 종류에 따라 파일명 뒤에 어떤 파일인지 표시하는 문자를 출력한다. ◇ / 디렉터리 ◇ * 실행파일 ◇ = 소켓파일 ◇ | FIFO ◇ @ 심볼릭 링크
-g	-l 옵션과 유사하지만 소유자 목록을 제외하고 출력한다.
-h	-l와 같은 파일의 크기를 보이는 옵션과 함께 쓸 때, 크기의 단위를 사용자가 읽기 쉬운 단위로 나타낸다.(k, m, g)
-H	-h 옵션과 비슷하나 용량의 기준단위를 1024로 한다.(1KB → 1024byte, 1MB → 1024KB)
-i	각각의 파일과 디렉터리 목록 앞에 i-node번호를 표시한다.

Linux 명령어: ls

- ls 명령 옵션

옵 션	내 용
-l	각각의 파일과 디렉터리에 대해 다음과 같은 정보를 출력한다. ◇ 권한 ◇ 링크의 수 ◇ 소유자 ◇ 그룹 ◇ 크기 ◇ 마지막 갱신시간
-L	심볼릭 링크(Symbolic link)로 참조된 파일에 대한 정보를 출력한다.
-m	파일이나 디렉터리 사이에 , (콤마)로 구분하여 출력한다.
-n	-l 옵션과 비슷하지만, 사용자ID 대신 ID 번호와 그룹번호가 출력된다.
-o	-l 옵션과 비슷하지만, 그룹항목이 제외된다.
-p	디렉터리명 뒤에 /를 붙여 출력된다.(리눅스 7.2버전은 기본(default)으로 설정되어 있다. 보다 많은 정보를 보려면 -F 옵션이 유용하다.)
-q	v 파일명이 비텍스트인 것은 ?로 표기한다.
-r	파일이나 디렉터리의 정렬을 역으로 한다.
-R	서브디렉터리까지 재귀적으로 출력한다.
-s	파일명이나 디렉터리 앞에 512바이트블록 단위로 크기를 출력한다.
-t	파일의 출력순서를 최근 갱신시간 우선으로 출력한다.
-u	최근 접근시간(access time) 순으로 출력한다.
-x	파일의 정렬을 왼쪽에서 오른쪽으로 정렬된 칼럼으로 출력한다. 기본(default)은 위에서 아래로 정렬 후, 왼쪽에서 오른쪽으로의 정렬이다.

Linux 명령어: ls

- ls 명령 실행

```
$ ls  -l
total 24
drwxr-xr-x 2 dlee dlee 4096 7월 5 13:34 Desktop
drwxr-xr-x 2 dlee dlee 4096 7월 15 12:45 Papers
- rw-r--r-- 1 dlee dlee 1024 7월 15 10:27 notepad
- rw-r--r-- 1 dlee dlee  1024 7월 15 10:26 homework.hwp
- rwxr--r-- 1 dlee dlee  9643 7월 15 10:12 file1
- rwxr--r-- 1 dlee dlee 6673 7월 15 19:34 myinfo
```

문자	의 미
-	일반 파일
b	블록 특수 파일
C	문자 특수 파일
d	디렉터리 파일
l	심볼링 링크
P	파이프(FIFO)파일
s	소켓(Socket)파일

Linux 명령어: ls

- ls 명령: 와일드 캐릭터 사용

문 자	의 미
*	영 또는 연속적인 모든 문자에 대응
?	모든 한 문자에 대응
[ ]	문자의 영역 목록을 나타냄. [a-z]는 a부터 z까지를 의미하고, [a,z]는 a 또는 z를 의미
{ }	여러 개의 패턴을 쉼표로 분리해서 나타냄

```
$ ls p*
print_file      print_txt       ps_tool
```

```
$ ls c??
cnn   can   cmd
```

```
$ ls [a-c]*
audio_file      ac_dc           character       configure_file
```

```
$ ls {a,ch}*
audio_file      ac_dc           character
```

Linux 명령어: cd

- **cd dir**
  - 현재 작업 디렉토리를 dir 로 이동
  - 그냥 cd 만 치는 경우 홈 디렉토리로 이동 (또는 “$ cd ~”)
  - 예

```
$ cd /home/student

$ cd ../source

$ cd ..

$ cd
```

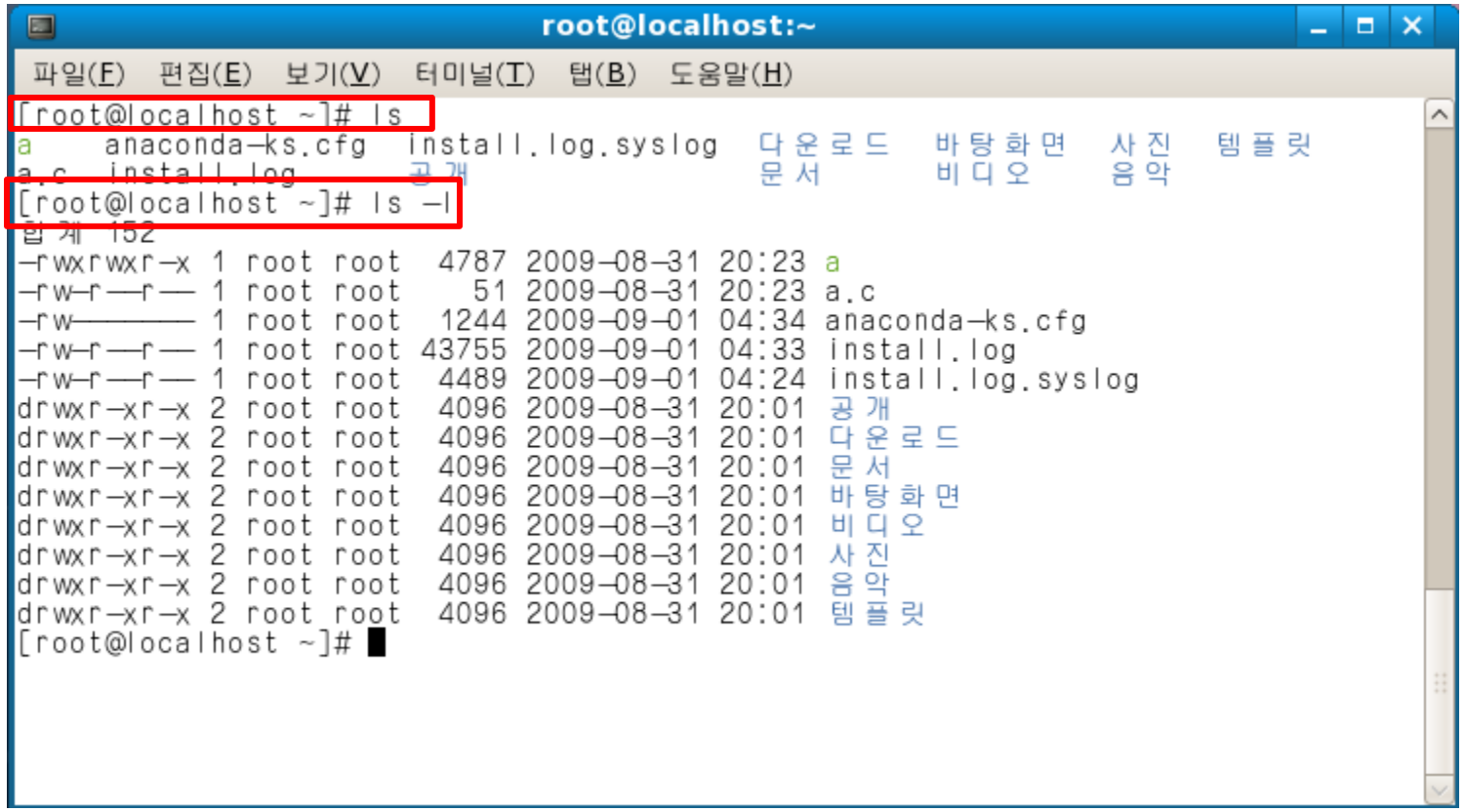
Linux 명령어: pwd

- **pwd: (print working dir) 약자**
  - 현재 작업 디렉토리 보기

```
$ pwd
/home/student/source
$ cd
$ pwd
/home/student
```

Linux 명령어 실습

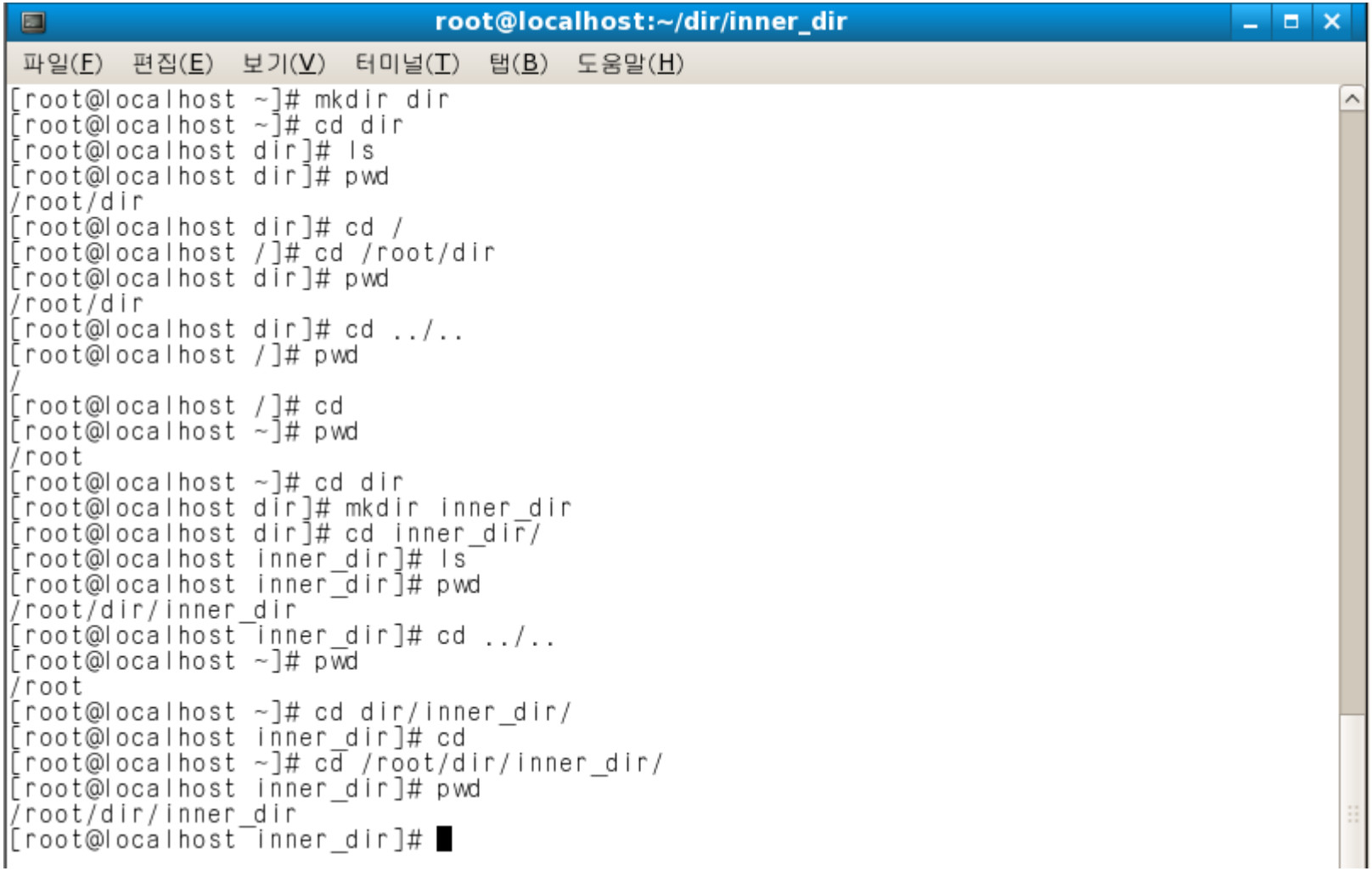
- ls 실습



권한, 하드링크 수, 사용자, 그룹, 크기, 수정날짜 및 시간, 이름

Linux 명령어 실습

- 디렉토리 관련 명령어 (cd, pwd) 실습



Copyright 2020. (Univ. of Seoul) All rights reserved.

- 수강생만 시청, 시청 후 삭제
- 변경, 복사, 배포 절대 금지

Linux 명령어: cat

- **$ cat**
  - 표준 입력 장치로부터 입력을 받아 표준 출력 장치로 출력 (필터 프로그램)
- **$ cat file**
  - file로부터 입력을 받아 표준 출력 장치로 출력

```
$ cat
Hello World
Hello World
                ⇦ ctrl + D
```

```
$ cat > file1
Hello World
⇦   ctrl + D

$ cat  file1
Hello World

$ cat  < file1
Hello World

$ cat  < file1  > file2
$ cat  file2
Hello World
```

이미 존재하는 파일에 추가하는 경우” >>” 사용

```
$ cat  >>  file1
BlaBlaBla  Ctrl+D

$ cat  file1
Hello World
BlaBlaBla
```

Linux 명령어 실습

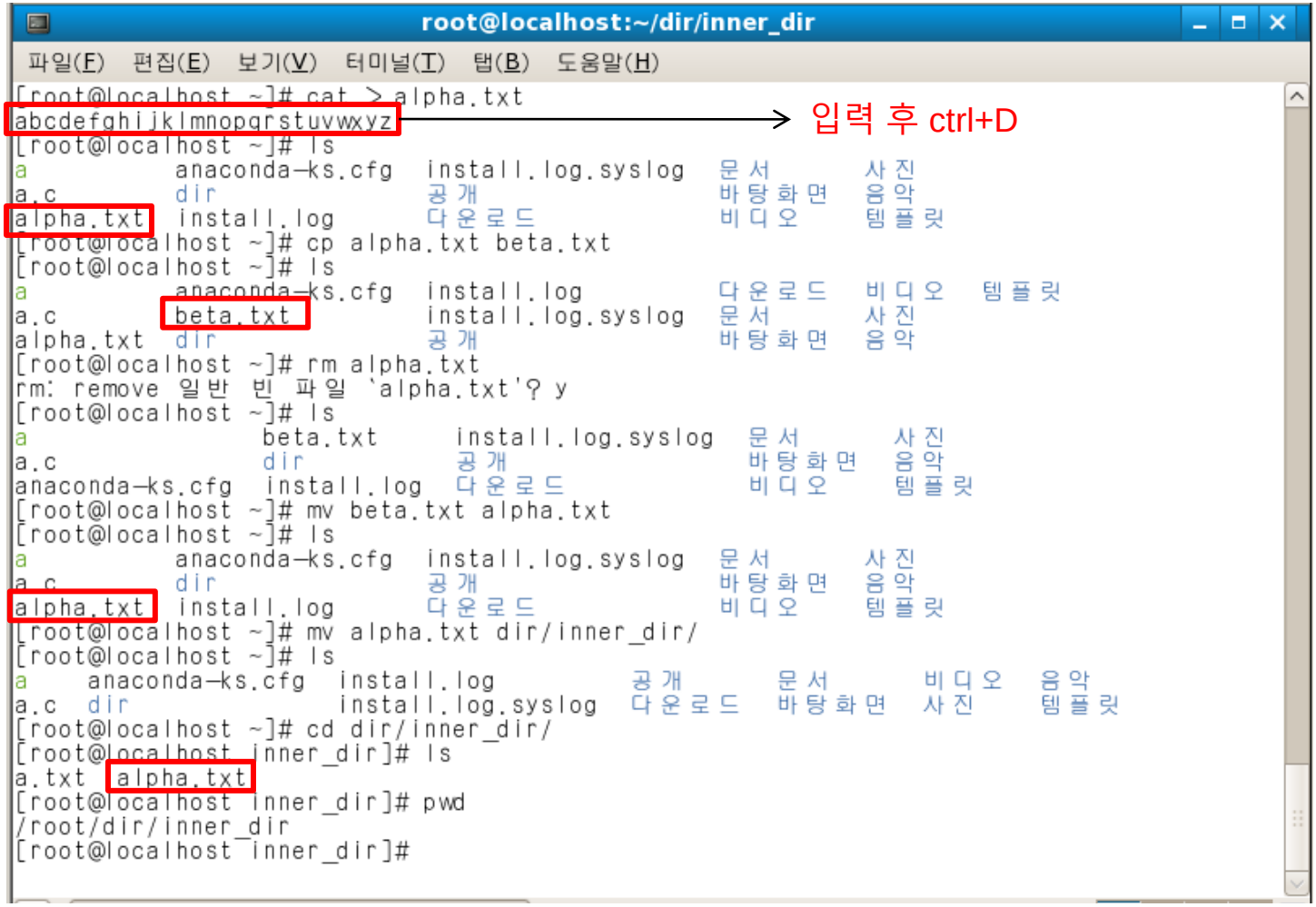
- **Cat 프로그램과 stdin, stdout, stderr**

```
/* main() 함수 호출 전에 3개 파일 open
   stdin = open("/dev/tty", O_RDONLY);
   stdout = open("/dev/tty", O_WRONLY | O_CREAT | O_TRUNC);
   stderr = open("/dev/tty", O_WRONLY | O_CREAT | O_TRUNC);
*/

main() {
   char  buf[BUF_SIZE];
   int     n;
   n = read(stdin, buf, BUF_SIZE);
   while (n > 0) {
        write(stdout, buf, n);
        n = read(stdin, buf, BUF_SIZE);
   }
}
```

Linux 명령어 실습

- **cat 명령어 실습**



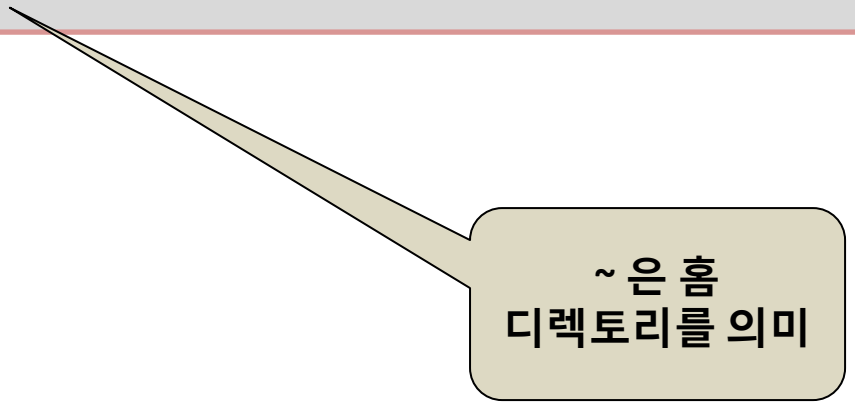
Linux 명령어: mkdir

- **mkdir dir1 [dir2] [dir3]**
  - 새로운 디렉토리 생성

```
$ mkdir  dir1   dir2
```

```
$ mkdir  /home/student/dir1  /home/student/dir2
```

```
$ mkdir  ~/dir1  ~/dir2
$ ls  ~
```



Linux 명령어: rmdir

- **rmdir dir**
  - 디렉토리 삭제
  - 디렉토리가 비어있지 않으면 삭제 불가능

```
$ rmdir  dir
```

  - 하위 디렉토리와 파일을 모두 삭제하고 싶으면 아래 명령 사용
    - $ rm –r dir

Linux 명령어: find

- **find**
  - 이름, 사이즈, 소유자, 날짜와 같은 특정 조건을 가진 파일 또는 디렉토리를 찾는 명령
  - -exec 옵션으로 찾아낸 파일에 대해 특정 명령을 실행시킬 수 있음
- **검색 옵션**

옵 션	내 용
-name	원하는 이름의 파일을 찾아 주며, 와일드카드(*, ?, [])를 사용할 수 있다.
-type	찾고자 하는 것이 파일(f)인지, 디렉터리(d)인지, 링크(l)인지 등을 정해준다.
-user	지정한 사용자가 소유한 파일을 찾는다.
-size	크기가 n인 파일을 검색한다. -n은 n보다 적음을 의미하고, n은 정확히 n을 의미하고, +n은 n 보다 큼을 의미한다. 이때 이 크기의 기준을 정하지 않으면 기본적으로 블록(b)으로 정하고, 바이트 단위(c)로 정할 수도 있고, kb 단위(k)로 할 수도 있으며, 워드(w) 단위로 할 수도 있다.
-newer	지정한 파일보다 나중에 생성된 파일을 찾아준다.
-empty	비어 있는 파일을 찾아낸다.

- **동작 옵션**

옵 션	내 용
-exec	찾아낸 파일을 인수로 지정한 명령을 수행한다.
-print	찾아낸 파일의 전체 경로를 표시해 보여준다. 이 옵션은 find 명령의 기본옵션이므로 굳이 써주지 않아도 된다.
-ok	'-exec' 옵션과 동일하나, 명령 수행 이전에 사용자에게 수행 여부를 확인한다.

Linux 명령어: find

- **이름으로 찾기**

```
$ find . -name "my*.*" -print
./test/myfav.txt
./student/u01/mynote.txt
```

- **파일 크기로 찾기**

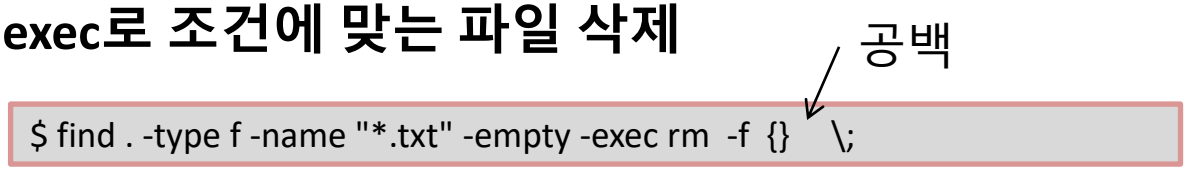
```
$ find . -name "my*.*" -print -size -2000c
./test/myfav.txt
```

-2000c는 2000바이트보다 작은 파일을 의미
-2k로 하면 2kilobyte보다 작은 파일을 의미

- **빈 파일 찾기**

```
$ find . -type f -name "*.txt" -empty
./test/b.txt
./test/c.txt
```

- **exec로 조건에 맞는 파일 삭제**



Linux 명령어: touch

- **touch : 파일의 수정 시간을 현재 시간으로 바꾸는 명령**

```
$ ls -l
total 32
drwxr-xr-x 2   dlee   dlee    4096    7월 4 13:23    Desktop
-rw-r--r-- 1    dlee    dlee    1024    7월 4 14:46    Papers
-rw-r--r-- 1    dlee    dlee    6875    7월 10 12:32    test.c
```

```
$ touch test.c
$ ls -l
total 32
drwxr-xr-x 2   dlee   dlee    4096    7월 4 13:23    Desktop
-rw-r--r-– 1   dlee    dlee    1024    7월 4 14:46    Papers
-rw-r--r-– 1   dlee    dlee    6875    9월 5 16:05    test.c      ⇦ 최근 수정시간 변경
```

Linux 명령어: cp

- **cp [option] file1 file2**
  - file1 내용을 file2로 복사
  - 만약 file2가 이미 존재하면 원래 file2 내용을 사라짐
- **cp [option] file1 dir**
  - file1을 dir로 복사
- **Options**

옵 션	기 능
-a	원본 파일의 속성, 링크 정보들을 그대로 유지하면서 복사한다. 이 옵션은 -dR 옵션과 같은 역할을 한다.
-b	복사할 대상이 이미 있어, 이것을 덮어쓰거나 지울 경우를 대비해 백업본을 만든다.
-d	만약 복사할 원본이 심볼릭 파일이면, cp 명령은 그 심볼릭 대상이 되는 파일을 복사한다.
-f	강제로 복사한다.
-i	만약 복사 대상 파일이 있으면 어떻게 처리할지 물어보는 프롬프트를 나타나게 한다.
-l	하드링크 형식으로 복사한다.
-P	원본 파일에 경로가 지정되어 있으면 그 경로 그대로 복사된다.
-p	원본 파일의 소유주, 그룹, 권한, 시간 정보들이 그대로 보존되어 복사된다.
-r	경로 안에 있는 모든 하위경로 파일들이 복사된다.
-u	소스파일이 기존파일보다 최신이거나, 기존파일이 사라졌을 때, 복사한다.
-x	원본과 대상 파일의 파일 시스템이 다르면 복사하지 않는다.

Linux 명령어: mv

- **mv [options] oldname newname**
  - 파일 또는 디렉토리 이름을 변경
- **mv [options] name dir**
  - “name” 이름의 파일 또는 디렉토리를 “dir” 디렉토리로 이동
- **Options**

옵 션	기 능
-b	원본 파일을 덮어쓰거나 지울 경우에 대비해 백업본을 만든다.
-f	대상 파일이 이미 있어도 사용자에게 어떻게 처리할지를 묻지 않는다.
-i	대상 파일이 이미 있어, 사용자에게 어떻게 처리할지를 물어 본다.
-u	대상 파일이 이미 있는데, 그 파일의 최근 변경시간이 원본 파일보다 최근의 것이면 파일을 옮기지 않는다.
-v	파일 옮기는 과정을 자세하게 보여준다.
-help	도움말을 보여준다.
-version	mv의 버전을 보여 준다.
-S	만약에 이동 대상이 이미 있어, 백업을 해야 할 경우에 그 백업 파일에서 사용할 파일 이름의 꼬리 문자를 지정한다. 이 옵션을 사용하지 않는다면 백업시에는 꼬리말은 ~이 된다.

Linux 명령어: rm

- **rm [options] name(or path)**
  - 파일/디렉토리 삭제
- **rm –r dir**
  - dir와 그 하위 디렉토리들을 모두 순회하면서 파일과 디렉토리를 모두 삭제
- **Otpions**

옵 션	기 능
-d	unlink와 함께 경로를 지운다. unlink하기 전에 그 경로가 비어 있는지 확인하지 않고, 그냥 unlink 해버린다. 이렇기 때문에, 만약에 그 지워지는 경로 안에 파일 있다면, 그 파일들의 종속성 문제가 생길 수 있다
-f	지울 파일이 없을 경우에 아무런 메시지를 보여주지 않고 그냥 넘어간다.
-i	각 파일을 하나씩 지울 것인지 사용자에게 전부 물어본다.
-r	하위 경로와 파일을 모두 지운다.
-v	각각의 파일 지우는 정보를 자세하게 모두 보여준다.
-help	도움말을 보인다.
-version	버전을 보인다.

Linux 명령어: chmod

- **chmod [mode] file/dir**

```
$ ls  -l
total 24
drwxr-xr-x 2 dlee dlee 4096 7월 5 13:34 Desktop
drwxr-xr-x 2 dlee dlee 4096 7월 15 12:45 Papers
- rw-r--r-- 1 dlee dlee 1024 7월 15 10:27 notepad
-rwxrwxr-- 1 dlee dlee  1024 7월 15 10:26 homework.hwp

$ chmod  644 homework.hwp

$ ls  -l
total 24
drwxr-xr-x 2 dlee dlee 4096 7월 5 13:34 Desktop
drwxr-xr-x 2 dlee dlee 4096 7월 15 12:45 Papers
- rw-r--r-- 1 dlee dlee 1024 7월 15 10:27 notepad
-rw-r--r-- 1 dlee dlee  1024 7월 15 10:26 homework.hwp
```

Linux 명령어: chmod

- **chmod 사용자 기호**

기 호	의 미
u (user)	소유자
g (group)	소유자가 속한 그룹
o (other)	그 외의 사용자
a(all)	모든 사용자

- **chmod 권한 기호**

기 호	의 미
+	권한 추가
-	권한 제거
=	명시된 권한 할당, 그 외는 제거 두 사용자의 권한을 같게 함

Linux 명령어: chmod

- **chmod 심볼릭 모드**
  - $ chmod ug+w test
  - $ chmod ug-w test
  - $ chmod a+wx test
  - $ chmod o=g test

Linux 명령어: umask

- **umask**
  - 파일을 생성할때 부여할 권한
  - 현재 umask 값 보기
    - $ umask
  - umask 설정
    - $ umask 022
  - File permisson 설정
    - FilePermission = Program설정Permission & ~umask

Linux 명령어: setuid, setgid

- **setuid, setgid**
  - 실행 가능 파일을 실행할 때 파일의 소유자 권한으로 실행
- **SetUid, SetGid 모드 설정 (설정 권한이 있는 경우에만 가능)**
  - setUid의 경우 8진수 값으로 4000
  - setGid의 경우 8진수 값으로 2000

```
$ ls -l
total 24
drwxr-xr-x 2   u9720541   student   4096 Sep 16 14:50   bin
-rw-r--r-- 1   u9720541   student      0 Sep 24 21:16   lecture.mpeg
-rwx------ 1   u9720541   student   5234 Sep 30 14:56   mbox          ⇦ 기본 권한 700

$ chmod 4700 mbox          ⇦ 기존 권한 700 + SetUid(4000)
$ ls -l
total 24
drwxr-xr-x 2   u9720541   student   4096 Sep 16 14:50   bin
-rw-r--r-- 1   u9720541   student      0 Sep 24 21:16   lecture.mpeg
-rws------ 1   u9720541   student   5234 Sep 30 14:56   mbox

$ chmod 2710 mbox          ⇦ 기존권한 710 + SetGid(2000)
$ ls -l
total 24
drwxr-xr-x 2   u9720541   student   4096 Sep 16 14:50   bin
-rw-r--r-- 1   u9720541   student      0 Sep 24 21:16   lecture.mpeg
-rwx--s--- 1   u9720541   student   5234 Sep 30 14:56   mbox
```

Linux 명령어: chown, chgrp

- **chown, chgrp:파일 소유자와 그룹소유자 변경**
  - 권한이 있는 경우에만 변경 가능

```
$ chown dlee test.c

$ chgrp cs_group test.c
```

Linux 명령어: ln

- **ln [options] filename newname**
  - 하드 링크 또는 심볼릭 링크 생성
- **Options**

옵션	의미	
-b	대상 파일이 이미 있으면, 그것을 백업한다.	
-d	디렉터리도 하드 링크가 가능하게 한다. 단, 이 옵션은 root의 권한을 가진 시스템 관리자 만 가능하다.	
-f	대상 파일이 이미 있어도 그냥 지운다.	
-I	대상 파일이 이미 있을 때 어떻게 할 것인지 사용자에게 물어본다.	
-n	만약 링크할 대상 파일이 이미 링크된 심볼릭 파일이면, ln 명령은 그 심볼릭 파일의 대상이 되는 원본 파일을 링크한다. 이렇게 하지 않고, 단지 그 심볼릭 파일 자체를 심볼릭 정보와 함께 링크하고 할 때, 이 옵션을 사용한다. 이 옵션은 --force 옵션과 함께 유용하게 쓰인다.	
-s	심볼릭(Symbolic) 링크를 한다. 소프트 링크라고도 한다.	
-v	각 파일의 작업 상태를 자세히 보여준다.	
--help	도움말을 출력한다.	
--version	버전 정보를 출력한다.	
-S	만약에 복사 대상이 이미 있어, 백업을 해야 할 경우에 그 백업 파일에서 사용할 파일 이름의 꼬리 문자를 지정한다. 이 옵션을 사용하지 않는다면 백업시에는 꼬리말은 ~이 된다.	
-V	백업하는 방법을 지정한다. 여기서 사용하는 백업방법은 GNU Emacs의 'version-control' 값과 같다. 아래와 같이 보다 짧은 지시어들도 사용될 수 있다. 여기서 사용될 수 있는 백업 방법은 아래와 같다.	
	t or numbered	항상 숫자 있는 백업본을 생성한다.
	nil or existing	대상 파일이 있을 경우에만 백업본을 생성한다.
	never or simple	간단한 백업을 생성한다.

Linux 명령어: ln

- **ln 실행 예**

```
$ ls -l file1
-rw-r--r--- 1    dlee    dlee    5000    7월 4  03:14    file1

$ ls -i  file1
14378  file1
$ ln   file1   file2
$ ls  -l  file1  file2
-rw-r--r--- 1    dlee    dlee    5000    7월 4  03:14    file1
-rw-r--r--- 1    dlee    dlee    5000    7월 4  03:17    file2
$ ls  -i  file1  file2
14378  file1      14378 file2
$ ln   -s   file1 linkfile
$ ls -i file1  linkfile
14378    file1     14394    linkfile

$ ls -l file1  linkfile
-rw-r--r--- 1    dlee    dlee    5000    7월 4  03:14    file1
lrw-r--r--- 1    dlee    dlee    5000    7월 4  03:19    linkfile -> file1
```

Linux 명령어: more

- **$ more txt_file**
  - 텍스트 파일을 화면단위로 나누어 출력
  - Space: 화면 단위로 다음 페이지 출력
  - Enter: 한 라인씩 출력
  - /: 검색하고자 하는 단어 입력, 검색된 페이지보기

옵 션	의 미	옵 션	의 미
-h	도움말	-/pattern	검색
-Space(=z)	다음 페이지	-=	현재 라인 출력
-Enter(=-1)	라인 스크롤	-!	명령어 실행
-d	반 페이지 스크롤	-^L	화면 다시 출력
-q	종료	-f	현재 파일명과 라인 출력
-f	다음 페이지	-number	number 줄을 한 화면에 지정한다
-b	이전 페이지		

Linux 명령어: less

- **$ less txt_file**
  - more와 유사하기 텍스트 파일을 화면단위로 출력
  - 앞과 뒤로 움직이면서 볼 수 있음

옵 션	의 미
-?	쓸 수 있는 하위 명령어를 보여 준다
-a	마지막 라인이 화면에 출력되고 나면 검색을 시작한다.
-c	전체 화면을 다시 갱신한다.
-e	두 번째로 파일의 끝에 도달하면 자동적으로 종료한다.
-E	파일의 끝에 도달하기만 하면 자동적으로 종료한다.
-i	대소문자를 구분하여 검색한다.
-n	행 번호를 추가한다.
-q	특정 에러가 발생하지 않으면 소리 없이 동작한다.
-Q	결코 아무 소리도 내지 않는다.
-s	연속되는 공백 라인은 하나의 행으로 처리한다.
-x number	number에 수치를 지정해서 탭 간격을 조정. 기본값은 8

Linux 명령어: cmp

- **$ cmp file1 file2**
  - File1과 file2을 비교하여 다른 점을 출력

옵 션	의 미
-l	다른 부분마다 바이트 번호는 10진수로, 다른 바이트는 8진수로 표시한다.
-s	아무런 결과물도 만들지 않고 하나의 값만을 되돌려서 표시한다. 0이면 동일이고, 1이면 다른 것, 2이면 비교 실패이다.

```
$ cmp file1   file2
file1  file2 differ: char 27, line 5
```

Linux 명령어: diff

- **$ diff file1 file2**
  - file1과 file2의 차이점을 줄 단위로 비교하여 출력

옵 션	의 미
-b	하나 이상의 Space 키나 Tab 키가 입력된 곳은 모두 하나로 간주한다.
-c LINE	파일이 생성된 날짜와 시간을 출력한다. LINE은 반드시 입력할 필요는 없다. 첫 번째 파일에만 있는 행은 -로 구성된 줄에 붙고 두 번째 파일에만 있는 행은 +가 붙는다. 두 파일에 다 있지만 다른 점이 있는 행은 !가 붙게 된다.
-d String	두 개의 파일이 혼합된 형식을 출력한다.
-e	ed의 명령어의 입력 자료로 쓰일 수 있는 출력물을 만든다.
-f	-e 플래그가 만들어 내는 출력물과 위아래 순서가 바뀐 출력물을 얻는다.
-i	대소문자를 구별하지 않는다.
-l	포맷된 출력 결과를 만든다. 요약도 덧붙여진다.
-n	-e 플래그와 위아래 순서가 반대되는 출력물을 만들고 더 있는 줄과 없는 줄의 수도 표시한다.
-r	지정해준 디렉토리들의 하위 디렉토리 중 같은 이름을 가진 것들이 있으면 모두에 대해서 diff 명령어를 실행시킨다.
-s	같은 이름을 가진 파일들 중에서 실제로 같은 파일들의 목록을 출력한다.
-S FILE	특정 파일을 무시한다.
-t	줄의 들여 쓰기를 보존해서 출력한다.
-w	모든 빈칸을 무시한다.

Linux 명령어: grep

- **grep**
  - 파일 안에서 특정한 패턴을 찾아 표준 출력 장치로 출력

옵 션	의 미
-b	찾은 각 라인의 앞에 블록 번호를 표시한다.
-c	찾은 라인들의 내용은 표시하지 않고 그 수만 세서 표시한다.
-C	일치하는 문자의 상하 두 라인까지 포함하여 출력한다.
-i	대소문자를 구분하지 않는다.
-l	결과를 출력할 때 지정한 패턴이 있는 파일의 이름만 출력한다.
-s	에러 메시지만을 출력한다.
-v	일치하지 않는 내용을 출력한다.
-w	한 단어로 일치해야 출력한다.
-x	전체 라인이 일치해야 출력한다.

```
$ ls | grep  a.hwp

$ find / -print  |  grep  myfile.c
```

Linux 명령어: head, tail

- **$ head txt_file**
  - txt_file의 앞 부분(디폴트로 10줄)을 표준 출력 장치로 출력
- **$ tail txt_file**
  - txt_file의 뒷 부분(디폴트로 10줄)을 표준 출력 장치로 출력
- **head 명령 옵션**

옵 션	의 미
-c number	파일의 처음 줄에서부터 number 만큼의 바이트를 출력한다.
-n number	파일의 처음 줄에서부터 number 만큼의 행을 출력한다.

- **tail 명령 옵션**

옵 션	의 미
-b number	지정한 수만큼 512 바이트 블록 다음부터 시작해서 출력한다.
-c number	지정한 수만큼의 문자에서 시작해서 출력한다.
-k number	지정한 수만큼 1024 바이트 블록 다음부터 시작해서 출력한다.
-n number	지정한 수의 행 번호부터 시작하여 출력한다.
-r number	파일의 내용을 뒤에서부터 시작하여 출력한다.
-f	파일의 크기가 변할 때마다 그 끝부분을 계속 출력한다.

Linux 명령어: tar

- **tar (Tape Archive)명령**
  - -c (생성) 옵션: 다수의 파일과 디렉토리 들을 한 파일로 묶음
  - -x (풀기) 옵션: 만들어진 tar 파일을 풀 때 사용
  - -z (압축) 옵션: 생성 또는 풀 때 Data Compress
- **사용법**
  - # tar –cvf [tar 파일 명] [디렉토리]
    - # tar -cvf tarfile.tar .
    - # tar -cvf tarfile.tar /mydir
  - # tar –xvf [tar 파일 명]
    - # tar –xvf tarfile.tar
  - -z 옵션으로 압축
    - # tar –zcvf tarfile.tar.gz .
    - # tar –zxvf tarfile.tar.gz

Linux 명령어: ps, pstree, jobs, who, bg, fg, kill

- ps: 실행중인 프로세스 보기
- pstree: 프로세스 트리 보기
- jobs: 실행중인 작업 보기
- who 또는 w: 로그인한 사용자 보기
- bg: 프로세스를 백그라운드로 보내기
- fg: 프로세스를 포어그라운드로 보내기
- kill: 프로세스 죽이기

새로운 터미널 창: xterm

- **X-term**
  - 새로운 터미널 프로그램 실행 (x-window 사용)
  - 백그라운드로 실행하도록 실행시 반드시 명령 줄 마지막에 '&' 사용할 것
    - $ xterm &

- **Xterm이 설치가 안된 경우 설치**
  - $ sudo apt install xterm

Runlevel

- **Linux runlevel**
  - /etc/inittab file is used to set the default run level for the system
  - /etc/rc[0-6].d: application files for each runlevel

- **Runlevel 변경**
  - $ init [0-6]

run Level	Mode
0	Halt
1	Single-User Mode
2	Multi-User Mode
3	Multi-User Mode with Networking
4	Undefined
5	X11
6	Reboot

환경변수

- **C언어에서 “char **environ” 문자열로 존재**

```
#include <stdio.h>
#include <stdlib.h>

extern char **environ;

int main() {}
```

- **환경변수 보기**
  - $ env
- **변경**
  - $ export PATH=.:$PATH
  - $ echo $PATH or $ env | grep PATH

Linux 텍스트 에디터

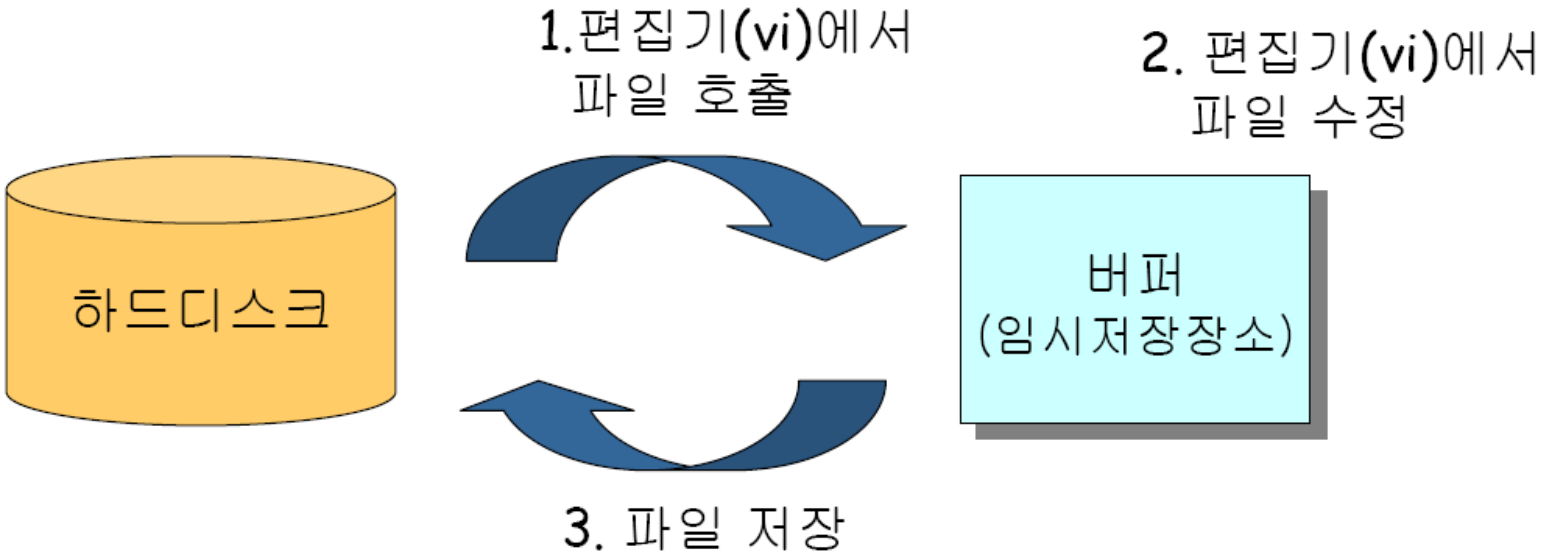
- **$ gedit file_name [&]**
  - Gnome 데스크 탑 환경을 위한 텍스트 편집기
  - UTF-8 호환 프로그램 코드, 스크립트 작성 및 편집을 위한 도구
  - 뒤에 & 를 붙이면 백그라운드로 실행
- **$ vi file_name**
  - Emacs와 함께 터미널을 사용하는 유닉스 환경에서 가장 많이 쓰이는 편집기
  - Vi(Visual Editor의 약자):빌 조이가 BSD 계열 유닉스를 위한 편집기로 개발
- **$ emacs file_name**
  - 리처드 스톨만이 매크로로 개발 (Editor MACroS의 약자)
  - 단순한 편집기 이상으로 통합 환경, 응용 프로그램 실행 환경

vi 편집기 사용법

vi 편집기

- **vi 편집기**
  - 버클리 의 BSD 유닉스 용으로 개발됨
  - System V의 기본 유틸리티로 채택
  - 실제로 모든 유닉스 시스템에서 사용됨
  - 리눅스 환경에서 가장 많이 사용하는 텍스트 편집기

vi 편집기

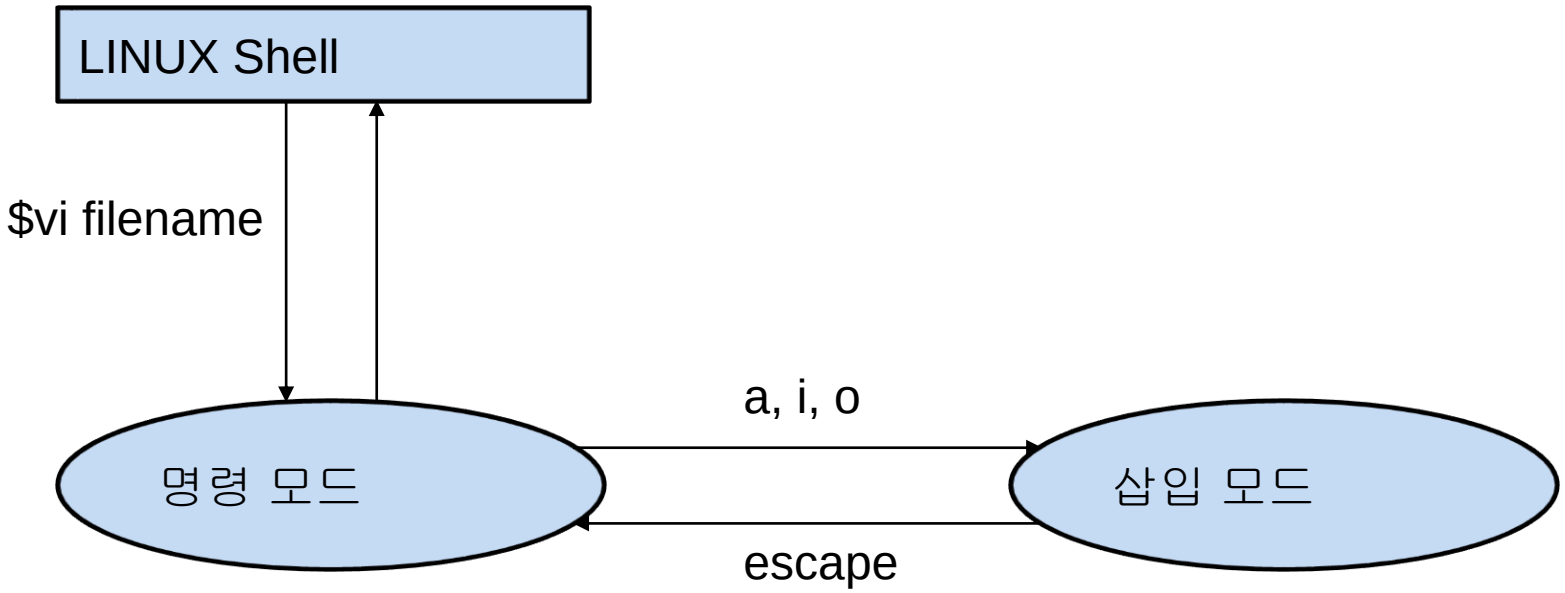


- **파일 내용을 잘못 수정한 경우 버퍼의 내용 무시**
- **작업 취소 기능 제공**

vi의 두 가지 모드

- **명령 모드(command mode)**
  - 편집기에 들어가면 명령어 모드에 놓이게 됨
  - 키보드를 통해 입력되는 문자는 편집기에 대한 명령어로 해석됨
  - 기록 명령어를 사용하여 변경된 파일의 내용을 저장하고 종료 명령어로 편집기를 빠져 나올 수 있음
- **삽입 모드(insert mode)**
  - 편집을 위한 명령어에 의해 텍스트 입력 모드에 들어가게 됨
  - 입력되는 모든 문자는 버퍼에 기록되어야 할 자료로서 받아들여짐
  - 명령어 모드로 가기 위한 특별한 명령어가 편집기마다 존재
  - 예) vi 편집기: Esc 키

vi의 두 가지 모드



vi 편집기 사용법

- **vi 편집기 모드**
  - 명령 모드
  - 텍스트 입력 모드
- **텍스트 입력 모드**
  - 다음 표에 나타난 키 중 하나를 눌러서 명령 모드에서 텍스트 모드로 전환

키	수 행
i	텍스트가 커서 앞에서 삽입된다
I	라인의 시작부분에서 삽입된다
a	텍스트가 커서 뒤에서 삽입된다
A	라인의 끝부분에서 삽입된다
o	텍스트가 현재 줄 다음부터 삽입된다
O	텍스트가 현재 줄 앞에서 삽입된다
R	텍스트가 대치된다(겹쳐 써진다)

vi 편집기 사용법

- **명령모드에서 검색, 수정 명령**
  - '/word' : word 검색
  - cw : 한 워드 변경
  - dw : 한 워드 삭제
  - x: 커서에 있는 글자 삭제
  - dd : 라인 삭제
  - 숫자dd : 숫자 만큼의 라인 삭제
  - yy : 라인 복사
  - 숫자yy : 숫자 만큼의 라인 복사
  - p : 붙여 넣기
  - u : undo
  - r : redo
  - n : 최근 명령 실행

vi 편집기 사용법

- **명령 라인 모드**
  - :q
    - 편집기 종료
  - :q!
    - 저장하지 않고 편집기 종료
  - :wq
    - 저장하고 편집기 종료
  - :숫자
    - 해당 줄로 이동
  - :s/orgword/newword/g
    - Orgword를 newword로 변경, 마지막 g는 옵션으로 파일에서 모든 orgword를 newword로 변경
  - : se nu
    - 라인 넘버 보여주기

vi 편집기 실습

- 1. 쉘에서 vi test.txt 를 입력
- 2. i를 눌러서 명령 모드에서 삽입 모드로 변경
- 3. 다음의 내용을 입력

```
When I find myself in times of trouble, mother Mary comes to me,
speaking words of wisdom, let it be.
And in my hour of darkness she is standing right in front of me,
speaking words of wisdom, let it be.

Let it be, let it be, let it be, let it be.
Whisper words of wisdom, let it be.
```

- 4. esc버튼을 눌러서 삽입 모드에서 명령 모드로 변경
- 5. :wq 키를 눌러서 데이터를 저장
- 6. 쉘에서 $ cat   test.txt 명령으로 파일의 내용을 확인

vi 편집기 설정

- **vi ~/.vimrc 입력**

```
set  cindent
set smartindent
set autoindent
set nowrap
set ruler
```

Makefile 유틸리티

- **3명이 함께 공동 프로그램을 작성한다고 가정**
  - User 1: file1.c 작성
  - User 2: file2.c 작성
  - User 3: file3.c 작성

  - 세 명 모두 common_header.h 공유

Makefile 유틸리티

- **User 1**

```
/* file1.c          */
/* 작성자: user1 */
#include “common_header.h”

int main()
{
    printf(“main₩n”);
    // 작업 1
    request_user2_func();
    // 작업 2
    request_user3_func();
    // 남은 작업
}
```

- **User 2**

```
/* file2.c          */
/* 작성자: user2 */
#include “common_header.h”

void request_user2_func()
{
    printf(“user2_func₩n”);
    // 요청 처리
   request_user3_func();
    // 응답
}
```

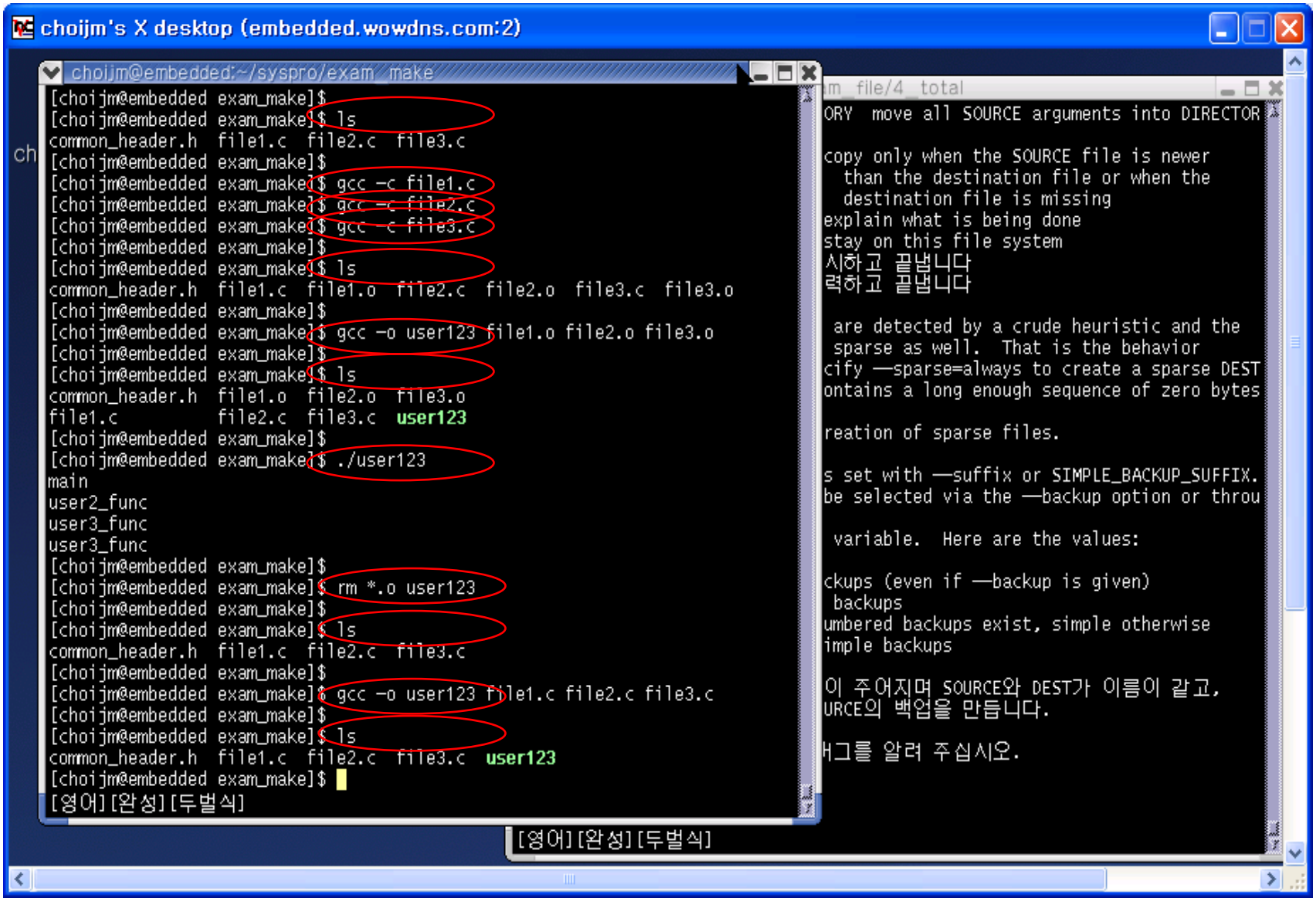
- **User 3**

```
/* file3.c          */
/* 작성자: user3 */
#include “common_header.h”

void request_user3_func()
{
    printf(“user3_func₩n”);
    // 요청 처리
    // 응답
}
```

Makefile 유틸리티

- 수행 파일 만드는 컴파일, 링크



Makefile 구조

- **make utility**
  - makefile(Makefile)에 정의된 순서대로 작업 수행
  - GNU 뿐만 아니라 많은 다른 개발 환경에서도 make 기능 지원 (hmake, armmake)
- **makefile 구조**
  - 목표(target), 의존관계(dependency), 명령(command)로 구성



Makefile 구조

- user1,2,3 예에 대한 makefile 작성

```
#
# makefile example for user123
#

all: user123

user123: file1.o file2.o file3.o
	gcc -o user123 file1.o file2.o file3.o

file1.o: file1.c common_header.h
	gcc -c file1.c

file2.o: file2.c common_header.h
	gcc -c file2.c

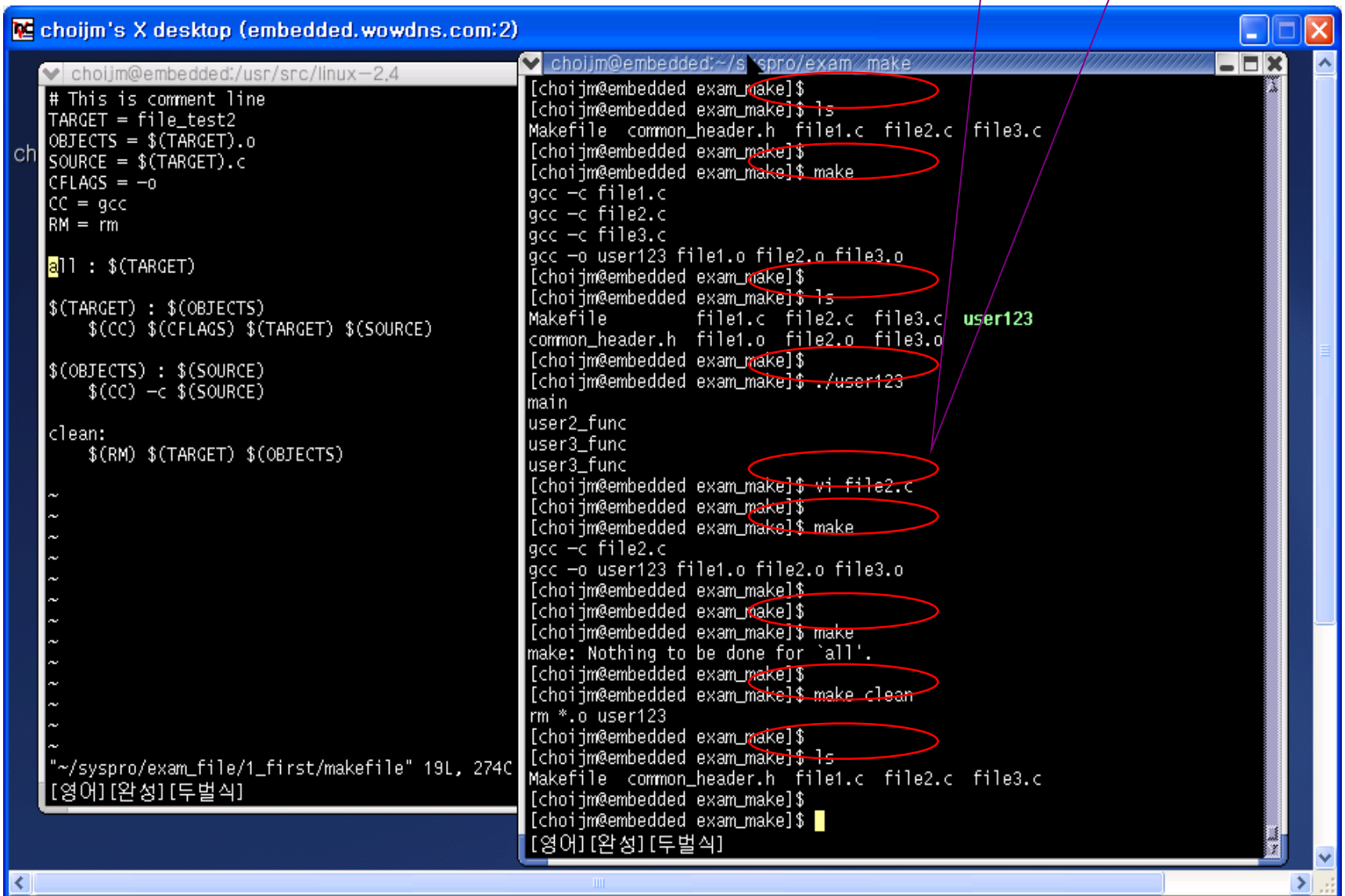
file3.o: file3.c common_header.h
	gcc -c file3.c

clean:
	rm *.o user123
```

Makefile 구조

- 수행 결과

vi common_ header.h 로 변경하면?



Makefile 구조

- **makefile 작성: Macro 이용**

```
#
# makefile example for user123 using macro
#
BUILD = user123
OBJECTS = file1.o file2.o file3.o
SOURCES = file1.c file2.c file3.c
HEADERS = common_header.h
CFLAGS = -c –Wall –O2
CC = cc

.PHONY: all clean
all: $(BUILD)

$(BUILD): $(OBJECTS)
    $(CC) -o $(TARGET) $(OBJECTS)

.c.o:
    $(CC) $(CFLAGS) –o $@  $<

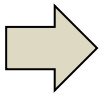
clean:
    rm $(TARGET) $(OBJECTS)
```

.SUFFIXES: .c. o

미리 정의된 매크로 보기

make –p | more

```
file1.o: file1.c $(HEADERS) Makefile
file2.o: file2.c $(HEADERS) Makefile
file3.o: file3.c $(HEADERS) Makefile
```



```
$(OBJS): $($@:.o=.c) $(HEADERS) Makefile
```

Makefile 구조

- **makefile 작성: dep 레이블 이용**

```
#
# makefile example for user123
#
BUILD = user123
OBJECTS = file1.o file2.o file3.o
SOURCES = file1.c file2.c file3.c
HEADERS = common_header.h
CFLAGS = -c –Wall –O2
CC = cc

.PHONY: all clean
all: $(BUILD)

$(BUILD): $(OBJECTS)
    $(CC) -o $(TARGET) $(OBJECTS)

.c.o:
    $(CC) $(CFLAGS) –o $@  $<

dep:
    gccmakedep $(SOURCES)

clean:
    rm $(TARGET) $(OBJECTS)
```

```
# DO NOT DELETE
file1.o: file1.c common_header.h /usr/include/unistd.h \
  /usr/include/features.h /usr/include/sys/cdefs.h \
  /usr/include/gnu/stubs.h /usr/include/bits/posix_opt.h \
  /usr/include/bits/types.h /usr/include/bits/wordsize.h \
  /usr/lib/gcc-lib/i386-redhat-linux/3.2.2/include/stddef.h \
  /usr/include/bits/typesizes.h /usr/include/bits/confname.h \
  /usr/include/getopt.h
```

Makefile 구조

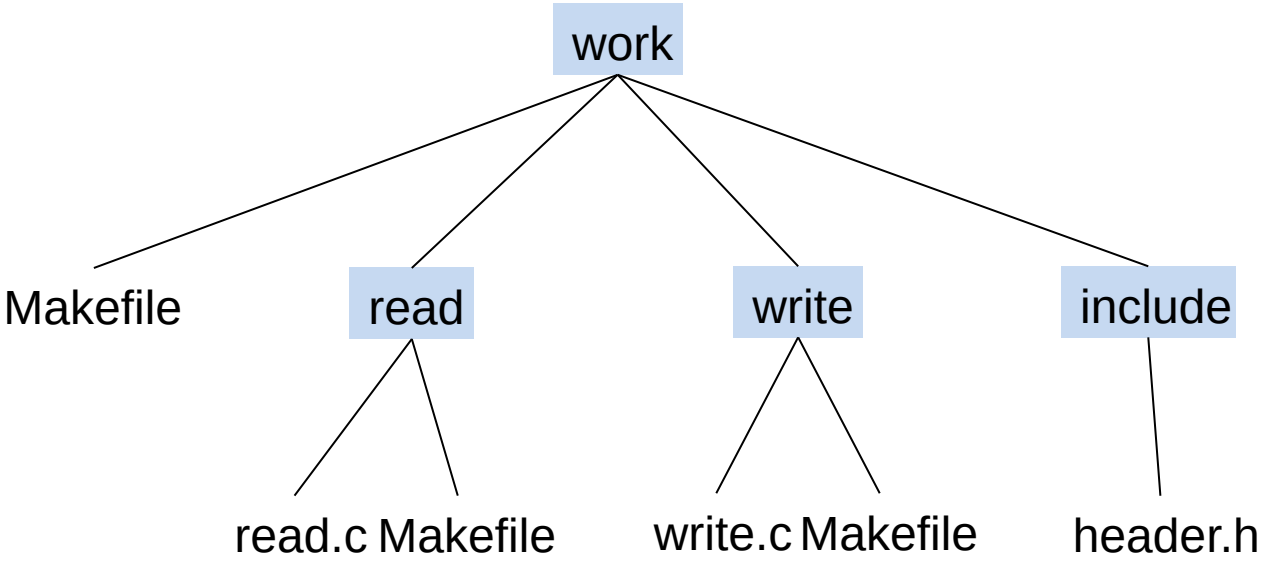
- **매크로 규칙**
  - predefined macro
    - CFLAGS += -g
    - CC = gcc
    - LDFLAGS = -shared
    - LD = ld
    - …
  - internal macro
    - $* : 확장자 없는 목표
    - $@ : 현재 목표
    - $< ,$? : 현재 목표보다 더 최근에 갱신된 파일 이름
  - 다중 target 가능 : client/server 컴파일
  - Recursive make : 커널 컴파일
  - Tex, Library 등을 위한 makefile도 가능

```
$(TARGET) : $(OBJECTS)
    $(CC) $(CFLAGS) -o $(TARGET) $(OBJECTS)
```

```
test.o : test.h
    gcc -c $*.c
```

```
.c.o :
    gcc -c $<
```

Recursive Make



Recursive Make

- work/read/Makefile

```
#
# read/Makefile
#

.SUFFIXES = .c .o

CC=gcc
CFLAGS= -g –c
INC=-I../include/

read.o: read.c

.c.o
    $(CC)  $(INC)  $(CFLAGS) $<

clean:
    rm  *.o
```

- work/write/Makefile

```
#
# write/Makefile
#

.SUFFIXES = .c .o

CC=gcc
CFLAGS= -g –c
INC=-I../include/

write.o: write.c

.c.o
    $(CC)  $(INC)  $(CFLAGS) $<

clean:
    rm  *.o
```

Recursive Make

- **work/Makefile**

```
.SUFFIXES = .c .o

CC=gcc
CFLAGS= -g –c
INC=-I./include/
TARGET=test
OBJ1=read.o
OBJ2=write.o
OBJ3=main.o
OBJECTS=$(OBJ1) $(OBJ2) $(OBJ3)

all: read write test

read: $(OBJ1)
    $(MAKE) –C read

write: $(OBJ2)
    $(MAKE) –C write

test: $(OBJ3)
    $(CC) –o $(TARGET) $(OBJECTS)
```

```
.c.o
    $(CC)  $(INC)  $(CFLAGS) $<

clean:
    $(MAKE) –C read clean
    $(MAKE) –C write clean
    rm  *.o
    rm  test
```